EXERCISE 19

RAP OData V4 UI Service — Flight Application

SAP S/4HANA Cloud ABAP / RAP Practice

Group 09 | *Package: ZS4D400_9309_RAP*

Item	Final Result
Exercise	19 — COMPLETED
Database Table	Z09FLIGHT
Root Entity	ZR_09FLIGHT
Projection	ZC_09FLIGHT
Service	ZUI_09FLIGHT_04
Binding	ZUI_09FLIGHT_04
Protocol	OData V4 - UI
Runtime Test	Fiori Elements Preview — Edit + Save

1. Task Explanation

Exercise 19 converts the Z09FLIGHT database table into a RAP transactional business object and exposes it as an OData V4 UI service. The final consumer is a SAP Fiori Elements application. The practical objective is to display flight records and successfully change a flight price.

```
Z09FLIGHT
   ↓
ZR_09FLIGHT       Root CDS / Business Object
   ↓
Behavior          CRUD + Draft + Validation
   ↓
ZC_09FLIGHT       Projection
   ↓
Service Definition
   ↓
Service Binding   OData V4 - UI
   ↓
Fiori Elements Preview
```

2. Development Objects

Object	Purpose
Z09FLIGHT	Persistent flight database table
ZR_09FLIGHT	RAP root business object data model
ZR_09FLIGHT behavior	Transactional rules and CRUD behavior
ZBP_R_09FLIGHT	Managed behavior implementation
ZC_09FLIGHT	UI/consumption projection
ZC_09FLIGHT behavior	Projection behavior
ZBP_C_09FLIGHT	Projection behavior implementation
ZUI_09FLIGHT_04	Service definition
ZUI_09FLIGHT_04	OData V4 - UI service binding
Z09FLIGHT_D	Draft table

3. Solution Method — RAP Generator

The repository-object generator was used because it creates the related RAP artifacts with consistent references. The sequence used was:

1. Select the Z09FLIGHT database table.

2. Choose Generate ABAP Repository Objects.

3. Select ABAP RESTful Application Programming Model → OData UI Service.

4. Assign package ZS4D400_9309_RAP.

5. Keep Z09FLIGHT as the referenced object.

6. Configure the root data model, behavior, projection, service definition and service binding.

7. Activate the generated objects.

8. Open the service binding and publish the local endpoint.

9. Choose Preview to launch Fiori Elements.

4. Behavior Definition — Final Logic

The behavior definition controls create, update and delete, draft handling, field restrictions, locking/ETag handling and the price validation.

```
managed implementation in class ZBP_R_09FLIGHT unique;
strict ( 2 );
with draft;
extensible;

define behavior for ZR_09FLIGHT alias Flight
persistent table Z09FLIGHT
draft table Z09FLIGHT_D
etag master LocalLastChangedAt
lock master total etag LastChangedAt
authorization master ( global )
{
  field ( mandatory : create )
    CarrierID,
    ConnectionID,
    FlightDate;

  field ( readonly )
    LocalCreatedBy,
    LocalCreatedAt,
    LocalLastChangedBy,
    LocalLastChangedAt,
    LastChangedAt;

  field ( readonly : update )
    CarrierID,
    ConnectionID,
    FlightDate;

  create;
```

```
    update;
    delete;

    field ( readonly ) PlaneTypeID;

    validation validatePrice on save
      { create; field Price; }

    draft action Activate optimized;
    draft action Discard;
    draft action Edit;
    draft action Resume;
    draft determine action Prepare;

    mapping for Z09FLIGHT corresponding extensible
    {
      CarrierID = CARRIER_ID;
      ConnectionID = CONNECTION_ID;
      FlightDate = FLIGHT_DATE;
      Price = PRICE;
      CurrencyCode = CURRENCY_CODE;
      PlaneTypeID = PLANE_TYPE_ID;
    }
  }
```

5. What the Behavior Statements Mean

Statement	Meaning
create	Allows a new flight to be created
update	Allows permitted fields to be changed
delete	Allows a flight to be deleted
with draft	Enables draft-based editing
mandatory : create	Field is required during creation
readonly	Field cannot be changed by the consumer
readonly : update	Field can be supplied initially but not changed later
validation	Runs a business check before save
etag	Helps prevent conflicting updates
lock	Controls concurrent access during transactions

6. Price Validation

Price is validated during save. The validation is declared in the behavior definition and its implementation is handled by the behavior implementation class.

```
validation validatePrice on save
  { create; field Price; }
```

The Fiori test also demonstrated datatype validation: entering 'a' into the numeric Flight Price field was rejected with the message 'Enter a number'.

7. Projection Behavior

The projection behavior exposes the operations of the underlying business object to the UI-facing projection. This is why the Fiori application can create, edit and delete through the service.

```
projection;
strict ( 2 );
```

```
define behavior for ZC_09FLIGHT alias Flight
{
  use create;
  use update;
  use delete;

  use action Edit;
  use action Activate;
  use action Discard;
  use action Resume;
  use action Prepare;
}
```

8. Service Definition and Binding

The service definition exposes the projection entity. The service binding then exposes that service through OData V4 - UI.

```
@EndUserText.label: 'Flight UI Service'
define service ZUI_09FLIGHT_04 {
  expose ZC_09FLIGHT as Flight;
}
```

Property	Value
Service Definition	ZUI_09FLIGHT_04
Service Binding	ZUI_09FLIGHT_04
Binding Type	OData V4 - UI
Entity Set	Flight
Endpoint	Published

9. Testing Steps

1. Open service binding ZUI_09FLIGHT_04.

2. Confirm the local service endpoint is published.

3. Select the Flight entity set.

4. Choose Preview.

5. Open a flight record.

6. Choose Edit.

7. Enter an invalid Flight Price such as 'a' and verify it is rejected.

8. Enter the valid numeric value 6000.

9. Save.

10. Verify the displayed price is 6,000.00 USD.

11. Verify the UI displays 'Object saved'.

10. Screenshot Evidence

These are the standalone screenshots available from the completed exercise session. They show the actual runtime verification and the one draft warning encountered during testing.

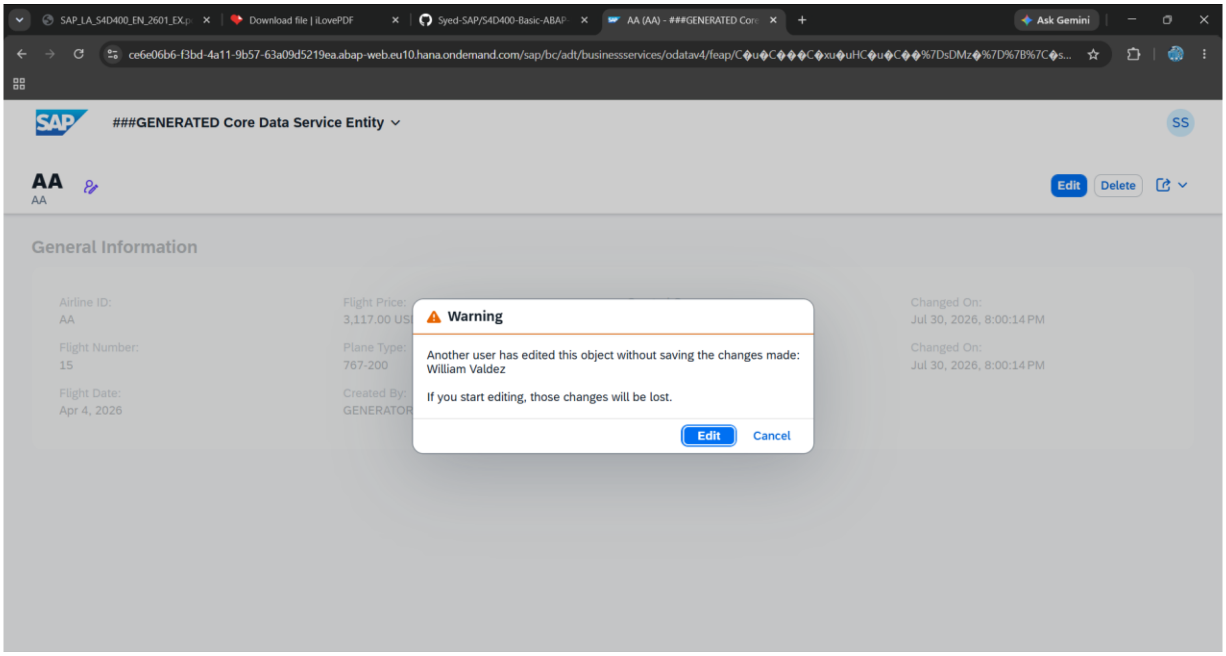

Figure 1 — Existing unsaved draft/session warning. This is not a RAP implementation error.

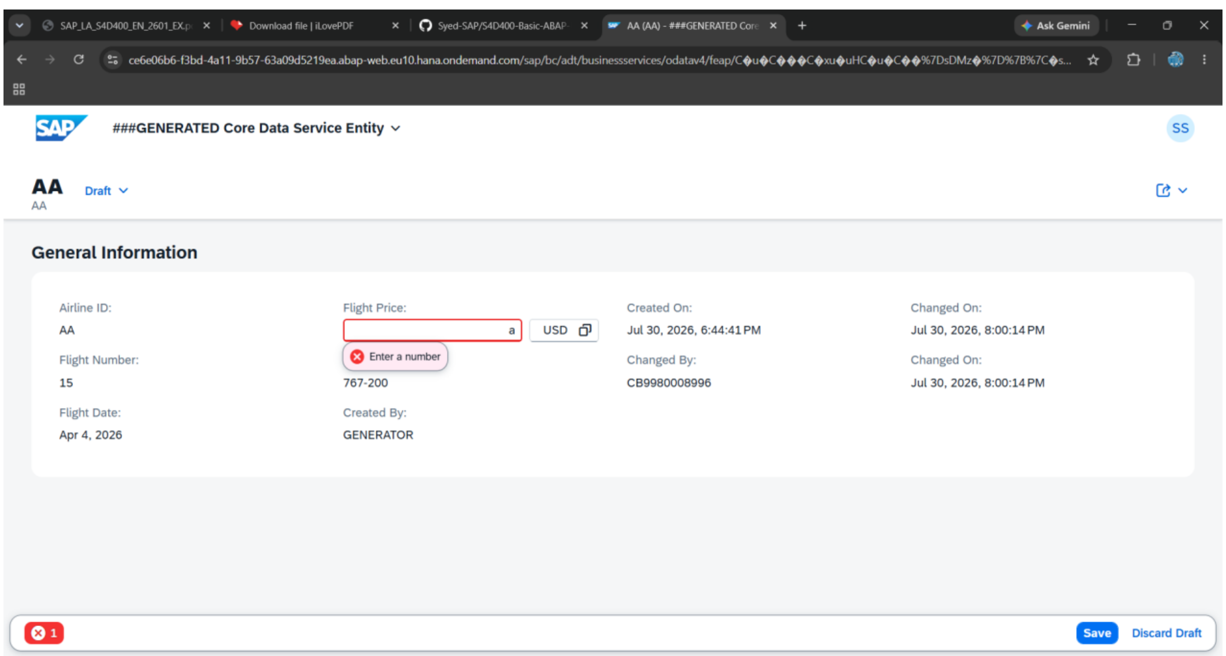

Figure 2 — Invalid Flight Price input rejected: “Enter a number”.

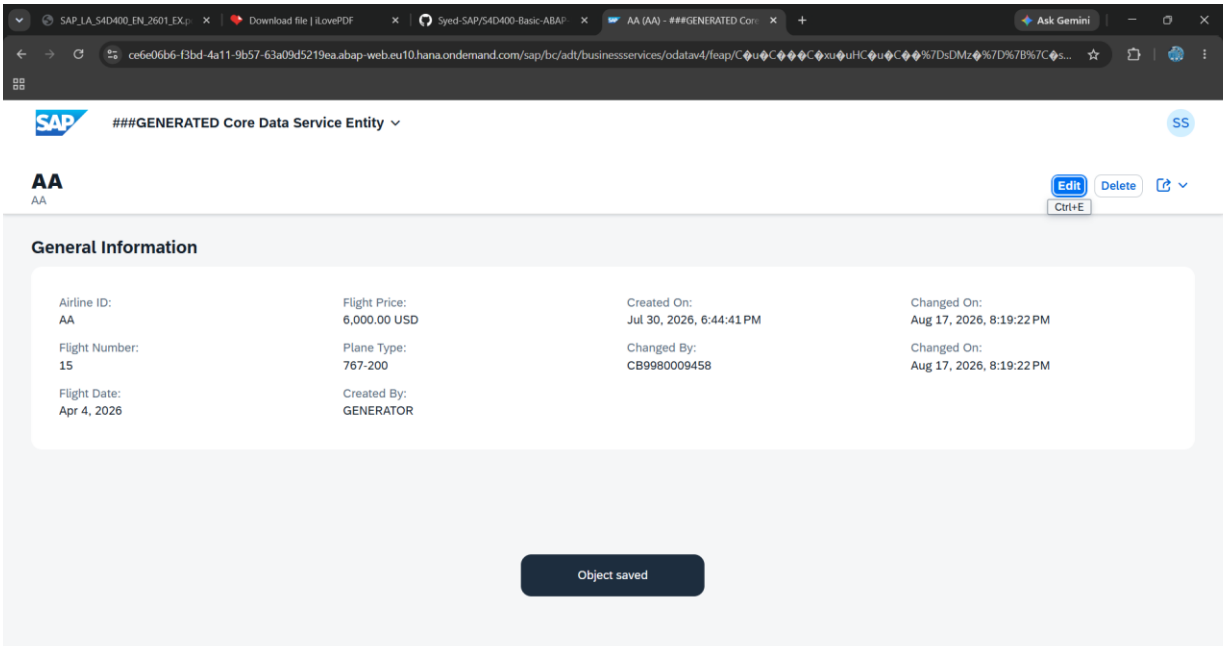

Figure 3 — Successful update: Flight Price = 6,000.00 USD and "Object saved".

11. Problems Encountered and Solutions

Issue	Solution
Generator said repository objects already existed	Reuse the existing objects or choose unique names. Do not create duplicate objects.
Behavior definition syntax errors	Correct the root behavior first and activate it before validating the projection/service.
Projection behavior errors on create/update/delete	Use the operations from the valid underlying behavior with use create/update/delete.
Service binding showed behavior syntax error	Fix and activate the underlying behavior definition, then reopen/refresh the binding.
Another user edited the object	Accept the draft warning and start your own edit session if the old unsaved changes are not needed.
Flight Price rejected 'a'	Correct behavior: Price is numeric. Enter a number.

12. Completion Checklist

☑ Z09FLIGHT available

☑ Root CDS entity ZR_09FLIGHT created

☑ Root behavior created and activated

☑ Managed behavior implementation ZBP_R_09FLIGHT created

☑ Draft support configured

☑ Create / Update / Delete enabled

☑ Price validation configured

☑ Projection ZC_09FLIGHT created

☑ Projection behavior created

☑ Service definition ZUI_09FLIGHT_04 created

☑ Service binding ZUI_09FLIGHT_04 created

☑ Binding type OData V4 - UI

☑ Service published

☑ Fiori Elements Preview opened

☑ Flight data displayed

☑ Invalid non-numeric price rejected

☑ Valid price 6000 accepted

☑ Object saved

☑ Saved value displayed as 6,000.00 USD

13. Final Result

EXERCISE 19 — COMPLETED

The complete RAP-to-OData-to-Fiori Elements flow is working. The final test changed the flight price to 6000 and successfully persisted it. The final screenshot shows 6,000.00 USD and the confirmation 'Object saved'.

14. Key Learning

• CDS defines the RAP data model.

• Behavior defines what the business object can do.

• Projection provides the controlled UI-facing layer.

• Service definition defines what is exposed.

• Service binding exposes the service through OData V4 - UI.

• Fiori Elements consumes the service without requiring a custom UI implementation.

• Draft, validation, locking and ETag handling are part of the RAP transactional model.

End of Exercise 19 Documentation